BREVE INTRODUCCION A LAS EXPRESIONES REGULARES EN C#

Autor: Héctor Fortún Castañón.

Lic. Cibernética-Matemática
Universidad de La Habana

Contenido:

Que son las expresiones regulares? 3

Como puedo validar una cadena con expresiones regulares bajo lenguaje C#? 4

Como se construye un patrón (pattern) de expresiones regulares en C#? 5

1. Uso de los corchetes (Ejemplo 1). 5

La negación dentro de corchetes. 9

2. Uso de las llaves (Ejemplo 2). 9

3. El uso del backslash \ 9

\t — Tecla TAB (Caracter de Tabulación). 11

\r — Caracter "Retorno del Carro" o "Car Return" (CR). 11

\n — Caracter "Nueva Línea" o Line Feed (LF)". 11

\a — Caracter "beep", o campana. 11

\e — Caracter "Esc" o "Escape" 11

\f — Caracter de "Salto de Página" o Form Feed (<FF>). 11

\v — Tabulador vertical (<VT>) 11

\xNN — Caracter ASCII hexadecimal 11

\uNNNN — Caracter Unicode 11

\d — Dígito decimal. 11

\D — Caracter cualquiera diferente de un dígito decimal. 12

\w — Carácter alfanumérico (alfabético o numérico incluyendo 12

el subrayado). 12

\W — Carácter NO alfanumérico. 12

\s — Espacio en blanco. 12

\S — Carácter diferente de un espacio. 12

\b — Límite de palabra. 12

\Z — Fin de la cadena 13

\B — Fuera de un límite de palabra. 13

\A — Inicio de la cadena 14
\G — Coincidencias en zonas contiguas de la cadena en análisis 14
\p — Clase de Caracter Unicode o de Bloque con Nombre 14
Categorías Generales Unicode 15
Bloques con Nombre (Tabla de Microsoft) 15
\P — Clase de Caracter de Negación de Unicode o de Bloque con Nombre 18
4. Construcciones de Alternancia (Uso del OR lógico "|") 18
5. Los Corchetes "[]" más en detalle 19
6. Los paréntesis "()" 20
7. El caracter "." 22
8. El signo de interrogación "?" 22
9. El formato (?:patron) 23
10. El formato (?=patron) 24
11. El formato (?!patron) 24
12. Los Cuantificadores 25
Cuantificadores expansivos y no expansivos 25
Algunos ejemplos del uso de cuantificadores (Microsoft) 26
13. Programas Utilitarios de Expresiones Regulares 27
14. El método Replace de la clase Regex. 28
Método Replace 28
15. Exclusión de Grupos de Caracteres de una Clase de Caracteres 34
16. Ejercicios a Resolver 34
16.1 - Ejercicio1 34
16.2 - Ejercicio2 34
16.3 - Ejercicio3 34
16.4 - Ejercicio4 34
16.5 - Ejercicio5 35
16.6 - Ejercicio6 35
16.7 - Ejercicio7 35
16.8 - Ejercicio8 35
16.9 - Ejercicio9 35
Resumen Final 36

Que son las expresiones regulares?

Cuando se observan determinadas cadenas de caracteres (strings), como por ejemplo, un número decimal, se nota que hay una determinada **ley de formación**, en este caso, una cadena de dígitos decimales seguida de un punto decimal y posteriormente otra cadena de dígitos decimales, en otro ejemplo, cuando se observa una fecha pues imaginamos rápidamente una cadena de la forma **dd/mm/aaaa** o **mm/dd/aaaa** donde **dd** representa el día, **mm** el mes y **aaaa** los 4 dígitos del año, en fin, estamos en presencia de una **determinada regularidad en su ley de formación,** de donde se infiere que si se crea un patrón (pattern en la literatura inglesa) con una determinada sintáxis que pueda describir dicha ley de formación, pues esto facilitaría considerablemente algunos procesos como las validaciones, los reemplazos etc. dentro de procesos computacionales, y exactamente esto es lo que se le ha dado en llamar **expresiones regulares** las cuales existen en varios lenguajes de programación, como **C#, C++, Visual Basic, Php, Oracle, MySQL,** y otros muchos más**.**

Las expresiones regulares son patrones de validación que se representan mediante una cadena de caracteres parametrizada donde cada caracter tiene un significado dentro del proceso de identificación de subcadenas, por tal razón, cuando definimos un determinado patrón, este podría **aceptar** o **no,** a una cadena de caracteres de entrada.

Existen dos categorías generales de expresiones regulares: POSIX y PERL.

En nuestro caso centraremos la atención en el segundo grupo (PERL), que es la familia a la que pertenece el motor de expresiones regulares de .NET que usa el C# de Visual Studio. Si el lector desea profundizar teóricamente en este tema, podrá encontrar mucha información publicada en la Internet aunque también el HELP de Visual Studio será una magnífica referencia.

En la medida en que se profundice teóricamente en este tema el lector se percatará rápidamente del ahorro de código que se puede obtener en los procesos de validación, reemplazo y otros con el uso de estas expresiones.

El objetivo fundamental de este documento es permitirle al neófito en este tema que pueda romper la inercia con rapidez a fin de poder utilizar estas técnicas en la programación bajo lenguaje C# de Microsoft, también es conveniente aclarar que con tal fin creamos este sencillo programa utilitario <RegexTest> que le permitirá procesar las expresiones regulares que usted defina, generando de paso, el código C# correspondiente a las validaciones, reemplazos y SPLITS que desee para su programa, lo cual le ahorrará mucho tiempo de programación en su tarea.

Como puedo validar una cadena con expresiones regulares bajo lenguaje C#?

Muy sencillo, existe una clase denominada Regex que nos permitirá realizar estas acciones, siga los siguientes pasos:

- Declare una sentencia: `using System.Text.RegularExpressions;` en el encabezado del programa.

- Instancie un objeto con esta clase especificando en el constructor el patrón de validación bajo el formato general que se muestra a continuación:

  ```
  Regex ObjetoRegex = new Regex(@"patrón de validación");
  ```

  * Es recomendable usar la arroba para el **string** del patrón de validación a fin de poder especificar algunos caracteres especiales que pueden ser requeridos.

Observemos a continuación una validación hecha mediante el objeto `ObjetoRegex` instanciado en el paso anterior y bajo el uso del método booleano **IsMatch** que chequea si la cadena de entrada que aparece en la propiedad **Text** del control denominado **TextBox1** sin los espacios a la izquierda o a la derecha, cumple los requisitos dados en el patrón:

```
if  (!ObjetoRegex.IsMatch(TextBox1.Text.Trim()))
    {
    MessageBox.Show("Cadena Erronea", "<ERROR>", MessageBoxButtons.OK,
    MessageBoxIcon.Error);
    }
else
    {
    MessageBox.Show("Cadena Correcta", "<OK>", MessageBoxButtons.OK,
    MessageBoxIcon.Information);
    }
```

Como se construye un patrón (pattern) de expresiones regulares en C#?

Dado que la cantidad de parámetros existentes es bastante extensa, y como siempre sucede, existiran algunas validaciones que nos van a interesar más que otras, pues centraremos la atención en las más usadas las cuales se explicarán mediante ejemplos prácticos que es la mejor forma de captar con rapidez cualquier concepto. A partir de este momento mostraremos varios ejemplos prácticos que le ayudarán a captar con rapidez los conceptos expresados.

1. Uso de los corchetes (Ejemplo 1).

Hemos comenzado a propósito con el uso de los corchetes y su significado dentro de un pattern o patrón de expresiones regulares.

Cada corchete de la forma [cadena_patron] representa una posición de carácter en la cadena de entrada en análisis, y en cadena_patrón se dice <que caracteres> pueden aparecer en esa posición, así, por ejemplo:

[13][5-9][ABX] Indica que se aceptan <tríos de caracteres> cuyo primer carácter tiene que ser un 1 o un 3 (nada más; observe que dentro de los corchetes los caracteres no se separan con comas, pues la coma sería un carácter más de los aceptados), que el segundo carácter de la cadena de entrada tiene que ser un dígito cualquiera entre el 5 y el 9, y que el tercer carácter de la cadena de entrada debe ser una A, una B o una X. El motor de expresiones regulares analiza en nuestro caso las ternas de caracteres y cuando finaliza la primera, entonces continúa mirando al frente buscando si existen más ternas que cumplan la condición, como en el segundo y tercer ejemplo.

De lo anterior se infiere que serán aceptadas las siguientes cadenas para este patrón:

37X ==> Cadena aceptada con número de coincidencias igual a 1.

29B15A==> Cadena aceptada con número de coincidencias igual a 1 (15A), pues en 29B el primer carácter no es ni 1, ni 3.

15A38X16Z ==> Cadena aceptada con número de coincidencias igual a 2, pues la última terna tiene bien los dos primeros caracteres pero la Z no está en el rango especificado por el patrón de entrada.

25A ==> Cadena no aceptada con número de coincidencias igual a 0 pues el primer carácter no es ni 1, ni 3.

*** Es necesario aclarar que la cadena que representa al patrón de validación no debe contener espacios al inicio o al final pues esto puede provocar errores sutiles.**

Si suponemos ahora el siguiente patrón:

[0-5]

Esto quiere decir que cada carácter de la cadena de entrada solamente puede

contener dígitos del 0 al 5 solamente para que exista coincidencia en esa posición, por tal razón:

La cadena "14205" tiene 5 coincidencias y es aceptada dado que al menos existe una coincidencia.

La cadena "14905" tiene 4 coincidencias y es aceptada dado que al menos existe una coincidencia.

La cadena "78" no tiene coincidencias y por tanto no es aceptada.

Nota:

Le sugerimos que pruebe estos ejemplos con el programa utilitario adjunto "RegexTest".

Después de haber visto el primer tipo de especificación de pattern de expresiones regulares, posiblemente usted se esté diciendo a sí mismo, todo muy bonito, pero como lo hago por programa, pues bien, a continuación, le dedicaremos unos párrafos a este tema tan importante y verá que lo comprenderá todo con facilidad, pues este documento está fundamentalmente orientado a la práctica.

Primeramente, se debe tener en consideración que existe en C# una clase denominada **Regex** que el programador puede utilizar a fin de instanciar objetos de expresiones regulares, la misma requiere que el patrón se especifique como un parámetro del constructor. Recuerde de especificar al inicio del programa la sentencia:

```
using System.Text.RegularExpressions;
```

Ejemplo:

```
Regex ObjetoRegex = new Regex(@"[0-5]");
```

*** En este ejemplo se ha instanciado un objeto denominado ObjetoRegex cuyo patrón nos indica que serán aceptadas las cadenas de caracteres que contengan al menos un dígito del 0 al 5. En otras palabras, esto indica que cada vez que se haga referencia a `ObjetoRegex` estaremos buscando dígitos del 0 al 5 dentro de la cadena que se analice.**

Por lo dicho anteriormente si tenemos que la variable de string Var = "12434", se podría escribir el siguiente código C# a fin de validarla con el pattern anterior y haciendo referencia al objeto instanciado en el párrafo anterior `(ObjetoRegex)`:

```
if  (ObjetoRegex.IsMatch(Var))
    {
    // Se debe observar que al referenciar a ObjetoRegex ya estamos
    // diciendo al motor de expresiones regulares que nuestro patrón
    // es [0-5] tal y como lo definimos en la instanciación del objeto.
    // Hay match al menos una vez en Var y el número de coincidencias
    // queda en la variable entera <Coincidencias> cuyo valor es
    // retornado por la propiedad <Count> del método <Matches>.
    int Coincidencias = ObjetoRegex.Matches(Var).Count;
    // El resultado final es Coincidencias = 5
    . . .
    }

else
    {
    // Por aquí se entra si no hubo absolutamente ninguna coincidencia
    // y se escribirá el código correspondiente a tal condición.
    . . .
    }
```

Existe otra forma de invocar a los métodos **IsMatch** y **Matches** sin necesidad de instanciar un objeto y es que la clase Regex tiene también métodos estáticos (se llaman con el nombre de la propia clase, Regex) que reciben dos parámetros; **el primero** contiene la variable a analizar **y el segundo** el patrón de validación, de tal forma:

```
ObjetoRegex.IsMatch(Var)
```
sería equivalente a la expresión:
```
Regex.IsMatch(Var, @"[0-5]")
```

y para determinar las coincidencias:
```
ObjetoRegex.Matches(Var).Count;
```
sería equivalente a la expresión:
```
Regex.Matches(Var, @"[0-5]").Count;
```

Por lo dicho anteriormente el código del ejemplo anterior quedaría en la forma siguiente bajo el nuevo formato:

```
if    (Regex.IsMatch(Var,@"[0-5]"))
      {
      int Coincidencias = Regex.Matches(Var,@"[0-5]").Count;
      . . .
      }
else
      {
      // Por aquí se entra si no hubo absolutamente ninguna coincidencia
      // y se escribirá el código correspondiente a tal condición.
      . . .
      }
```

* Se observará que se está utilizando el símbolo de @ en las cadenas de caracteres, de esta forma podrán aparecer secuencias de escape como los slash y otras más, dentro de la cadena en cuestión.

De ahora en adelante haremos referencia a cualquiera de los dos formatos anteriores y usted debe estar familiarizado con los mismos.

Después de haber visto todo lo anteriormente explicado veremos otro formato de expresiones regulares, se trata del uso de los caracteres **^** y **$** al inicio y al final del patrón, es decir, si escribimos `@"^[0-5]$"` estamos diciendo que el análisis se hará obligatoriamente desde el inicio de la cadena ("**^**") y que dicha cadena estará compuesta solamente por un caracter que debe ser un dígito entre el 0 y el 5 lo cual estará implícito por la aparición del símbolo ("**$**"). En otras palabras, estamos diciendo que se espera una entrada de un solo caracter y que el mismo debe ser un dígito entre 0 y 5. (Detalle: $ también acepta un salto de línea \n al final de la cadena; para una validación estricta se puede usar \A al inicio y \z al final.)

Cuando los corchetes estan comprendidos entre estos dos símbolos especiales **(^** y **$)** cada par de corchetes representa un caracter con presencia obligatoria en la cadena de entrada, que además debe estar en el rango especificado dentro de dicho corchete.

Como ejemplo de lo anteriormente dicho si suponemos el patrón:

```
@"^[0-5][8][479]$"
```

se esta asumiendo que solamente se aceptaran cadenas de tres dígitos obligatorios donde el primer caracter tiene que ser un dígito del 0 al 5, el segundo un 8 y el tercero un dígito que tiene que ser un 4, un 7 o un 9, de esta forma se aceptarian las cadenas siguientes:

- 587
- 084
- 489
- 184

En contraposición, no se aceptarían:

- 5875 ==> Tiene más de tres caracteres
- 587084 ==> Tiene más de tres caracteres
- 579 ==> El segundo dígito no es un 8
- 111 ==> El segundo y el tercer dígito está fuera de rango

Ahora, posiblemente el lector esté analizando otras alternativas, como, por ejemplo, que sucedería si quitamos el símbolo ^ del patrón, es decir, [0-5][8][479]$, pues bien, en este caso estamos diciendo al motor de análisis de expresiones regulares que el análisis no

comenzará obligatoriamente al inicio de la cadena sino que se escogerían los últimos tres caracteres.
Atendiendo a lo anterior la cadena "587084" nos daría que la coincidencia es la subcadena "084", y si ponemos la cadena "587" pues la coincidencia sería esa misma cadena.

Y que sucedería si quitamos el símbolo $ del patrón anterior, es decir, [0-5][8][479],
muy sencillo, ahora le estamos diciendo al motor de expresiones regulares que se aceptaran todas las subcadenas dentro de la cadena de entrada que cumplan la condición dada por dicho patrón, por ejemplo:
La cadena "587084" nos daría dos coincidencias, la subcadena "587" y la subcadena "084" ya que la ausencia del símbolo $ nos está quitando la cota de la derecha.

La negación dentro de corchetes.

Es posible negar las especificaciones dadas dentro de un corchete y el formato es el siguiente:

[^especificación]

Es decir, si se especifica [**^**6], esto quiere decir que en esa posición se acepta cualquier caracter que no sea un dígito 6. Como ejemplo práctico, si no queremos que exista ningun dígito decimal en esa posición escribiriamos [**^**0-9].

2. Uso de las llaves (Ejemplo 2).

Las llaves se interpretan como número de caracteres a analizar en la cadena de entrada, así, por ejemplo:

El patrón "[5-6]{3}" nos indica que se aceptaran solamente subcadenas cuyos dígitos esten entre 5 y 6 y que obligatoriamente tengan exactamente 3 caracteres de longitud, asi, la cadena "565084665" nos daría dos coincidencias, las subcadena "565" y la subcadena "665" dado que la "084" está fuera de rango.
Se debe observar que el patrón "[5-6]{3}" es equivalente al patrón "[5-6][5-6][5-6]" escrito en forma reducida.

3. El uso del backslash \

El caracter **"\"** es frecuentemente utilizado para definir diferentes secuencias de escape con un significado dado, por ejemplo, cuando se desea indicar que en una posición dada de la cadena de entrada debe haber un dígito decimal que hasta el momento lo hemos estado simbolizando mediante corchetes "**[**0-9**]**", pues esto también se puede indicar mediante un **\d,** por tal razón, un patrón de la forma siguiente:

"^[0-9]{3}\.[0-9]{2}$" equivaldría a ==> "^\d{3}\.\d{2}$"

nos está diciendo que solamente se aceptan cadenas de 6 caracteres que representan números de tres dígitos enteros, seguidos de un punto decimal y de dos dígitos decimales. Ejemplo: 013.22 (ver la nota sobre \d más adelante).

A forma de ejemplo, si queremos validar cadenas que deben tener al inicio tres dígitos decimales seguidos de un guión y 4 dígitos decimales pues escribiríamos el siguiente patrón:

^\d{3}\-\d{4}$

lo cual equivale también a este otro patrón bajo el uso de corchetes:

^[0-9]{3}[-][0-9]{4}$

Algunos ejemplos:

Cadenas Aceptadas	Cadenas No Aceptadas
832-0899	8323-0899 (tiene más de 3 dígitos antes del guión)
261-9130	261-91303 (tiene más de 4 dígitos después del guión)
838-0047	053-449005 (tiene más de 4 dígitos después del guión)

El metacaracter **"\"** indica que el siguiente caracter es especial, una literal o una secuencia de escape.

Por ejemplo, si se utiliza dentro del patrón el caracter "n", este representa a una letra **n** en la posición correspondiente, mientras que si se escribe "\n" el significado es completamente distinto pues define un caracter de cambio de línea (line feed).

Cuando se desea especificar un caracter no representativo ASCII mediante su valor hexadecimal interno pues se puede usar una secuencia de escape con el siguiente formato general:

\xNN

donde NN representa dos dígitos hexadecimales, de 00 a FF (la tabla ASCII y los caracteres latinos que le siguen, como la ñ = \xF1).

Por otra parte, si se desea especificar un caracter UNICODE cualquiera, se usa el formato siguiente:

\uNNNN

donde NNNN representa cuatro dígitos hexadecimales (el código UTF-16 del carácter).

Si se escribe:

\cX

esto se interpreta como un Ctrl-X dado que **\c** se asocia a la tecla **CTRL**.

Nota importante:

Cuando se desea incluir el backslash ("\") como caracter dentro del patrón tenemos que escribir "\\" y deja de ser un metacaracter para convertirse en un caracter ordinario dentro de la cadena.

Veremos a continuación una serie de secuencias de escape que pueden ser utilizadas dentro de expresiones regulares:

==========

\t — Tecla TAB (Caracter de Tabulación).

Ejemplo:

Cadenas Aceptadas	Cadenas No Aceptadas
3<tab>5	47<tab>56
8<tab>1	3<tab>55
9<tab>4	222<tab>888

\r — Caracter "Retorno del Carro" o "Car Return" (CR).
\n — Caracter "Nueva Línea" o Line Feed (LF)".
\a — Caracter "beep", o campana.
\e — Caracter "Esc" o "Escape"
\f — Caracter de "Salto de Página" o Form Feed (<FF>).
\v — Tabulador vertical (<VT>)
\xNN — Caracter ASCII hexadecimal
\uNNNN — Caracter Unicode

Idem al anterior pero para representar caracteres **Unicode,** por ejemplo "\u00A2" representa el símbolo de centavos.
* No todos los motores de Expresiones Regulares soportan Unicode.
El Framework .NET lo hace, pero otros pudieran no soportarlo.

\d — Dígito decimal.
*** Ya visto en los ejemplos anteriores. Es conveniente aclarar que en .NET \d acepta cualquier dígito decimal de Unicode, no solo del 0 al 9 (también, por ejemplo, los dígitos árabes ٠١٢٣); si se desean únicamente los dígitos del 0 al 9 se debe usar [0-9] o la opción RegexOptions.ECMAScript.

\D — Caracter cualquiera diferente de un dígito decimal.

*** Es el inverso del \d

\w — Carácter alfanumérico (alfabético o numérico incluyendo el subrayado).

En .NET acepta cualquier letra (incluidas la ñ, las vocales acentuadas y la ü), cualquier dígito decimal y el subrayado.

Solamente si se usa la opción RegexOptions.ECMAScript se corresponde con el patrón [A-Za-z0-9_], es decir, con el alfabeto inglés, que no incluye la ñ, las letras acentuadas, etc.

Por tanto, sin esa opción, \w ya sirve para palabras escritas en español.

\W — Carácter NO alfanumérico.

* Es el inverso del \w, lo que significa que puede ser cualquier caracter diferente de letras, números y subrayado (de cualquier alfabeto)

\s — Espacio en blanco.

*** Representa cualquier espacio en blanco: el espacio, el tabulador (\t), los saltos de línea (\n y \r), el salto de página (\f) y el tabulador vertical (\v), entre otros.

\S — Carácter diferente de un espacio.

*** Representa cualquier carácter que no sea un espacio en blanco.

\b — Límite de palabra.

Si tomamos el patrón siguiente: **er\b**

\b marca un límite de palabra, es decir, el principio o el final de una palabra. Por eso er\b indica que se aceptarán las cadenas que contengan una palabra terminada en "er", como por ejemplo, "amanecer", "tener", etc. Si se escribe delante, \ber, se buscan palabras que empiecen por "er", como "error".

Ejemplo:

Cadenas Aceptadas	Cadenas No Aceptadas
amanecer	error
prevalecer	persistir
mover	mover_

Sin embargo, si se pusiera el siguiente patrón: **^er\b$**

La única cadena aceptada sería "er" lo cual es muy diferente, pues el metacaracter ^ impone que antes de "er" esté solamente el inicio de cadena y el $ que después solo esté el final.

Nota aclaratoria:
Las secuencias de escape \b, \B, \G, \Z, \z, \A, ^, y $ que aparecen explicadas en este documento se les conoce como **DELIMITADORES** o también como **"Aserciones Atómicas de Ancho Cero"** las cuales hacen que la coincidencia se cumpla o no en dependencia de la posición que tenga la subcadena dentro de la cadena, pero no hacen que el motor de expresiones regulares avance en su análisis por la cadena ni consuma caracteres.

\Z — Fin de la cadena

Si tomamos el patrón siguiente: **er\Z**

Esto indica que la cadena aceptada será la que termine en "er": \Z marca el final de la cadena completa (o la posición justo antes de un salto de línea \n final).

Ejemplo:

Cadenas Aceptadas	Cadenas No Aceptadas
amar comer	error
prevalecer	persistir
partir amar mover	mover_

Observe que \Z solo mira el final de la cadena completa: en "comer\nhola" no hay coincidencia aunque la primera línea termine en "er". Si tampoco se quiere aceptar el salto de línea final, se usa \z (en minúscula), que exige el final exacto de la cadena.

\B — Fuera de un límite de palabra.

Si tomamos el patrón siguiente: **er\B**
\B es lo contrario de \b: la posición NO debe ser un límite de palabra. Por eso er\B acepta un "er" que no esté al final de una palabra, es decir, que vaya seguido de otra letra, como por ejemplo en "error". También se aceptarían "_error" o "persistir", pues en ambas el "er" va seguido de otra letra.

Ejemplo:

Cadenas Aceptadas	Cadenas No Aceptadas
error errata (En este caso hay 2 coincidencias)	mover
erradicar (En este caso hay 1 coincidencia)	comer y beber
error errata,erroneo:erratarorr (En este caso hay 5 coincidencias)	repeler

En la primera columna los "er" resaltados en amarillo provocan coincidencia. Si se pusiera el patrón ^er\B, solo se aceptaría un "er" situado al principio de la cadena y seguido de otra letra, como en "error".

\A — Inicio de la cadena

Si tomamos el patrón siguiente: \Aer
esto indica que se aceptarán las cadenas que empiecen por "er": \A marca el inicio de la cadena completa (no se debe escribir \A\er, pues \e representa el carácter ESC).

Por ejemplo, si tomamos la cadena "error errata Ernesto" provoca 1 coincidencia solamente, que se produce en el prefijo de la primera palabra (resaltado en amarillo).

\G — Coincidencias en zonas contiguas de la cadena en análisis

Si tomamos el patrón siguiente: **\G\(\d\)**

Esto indica que las cadenas aceptadas serán las que contengan la primera aparicion de subcadenas contiguas que sigan esta ley de formación definida por un paréntesis, un dígito y un paréntesis cerrado y que aparezcan a su vez como prefijo de la cadena en análisis.

Por ejemplo, si tomamos la cadena "(1)(9)(5)[2](4)(7)" las coincidencias aparecen resaltadas en amarillo, sin embargo, si tomamos la cadena "[2](5)(2)" esta no provoca coincidencias pues (5)(2) no aparecen como prefijos de la cadena en análisis.

\p — Clase de Caracter Unicode o de Bloque con Nombre

Coincide con cualquier carácter único que pertenezca a la categoría general Unicode o al bloque con nombre especificado por nombre.

Existen categorías generales de juegos de caracteres internacionales que agrupan un conjunto de caracteres especificos categorizados a los cuales se designa mediante un nombre que en las expresiones regulares debe estar entre llaves, es decir, bajo el formato {nombre_categoría} o también como {bloque_con_nombre}.

Cuando se desea que el patrón solo reconozca símbolos pertenecientes a las categorías anteriores o a los bloques con nombre pues entonces se debe utilizar la secuencia de escape \p bajo el formato que sigue:

\p{nombre_categoría}
\p{bloque_con_nombre}

==========

A continuación, mostramos en detalle las tablas extraidas de Internet (Microsoft)para ambos casos:

Categorías Generales Unicode

Categoría	Descripción
Lu	Letra, mayúscula
Ll	Letra, minúscula
Lt	Letra, inicial en mayúscula
Lm	Letra, modificador
Lo	Letra, otra
L	Todos los caracteres de letras. Esto incluye los caracteres Lu, Ll, Lt, Lm y Lo.
Mn	Marca, sin espacios
Mc	Marca, con espacios y combinación
Me	Marca, inclusión
M	Todas las marcas diacríticas. Esto incluye las categorías Mn, Mc y Me.
Nd	Número, dígito decimal
Nl	Número, letra
No	Número, otro
N	Todos los números. Esto incluye las categorías Nd, Nl y No.
Pc	Puntuación, conector
Pd	Puntuación, raya
Ps	Puntuación, abrir
Pe	Puntuación, cerrar
Pi	Puntuación, comilla de apertura (puede comportarse como Ps o Pe, en función del uso)
Pf	Puntuación, comilla de cierre (puede comportarse como Ps o Pe, en función del uso)
Po	Puntuación, otro
P	Todos los signos de puntuación. Esto incluye las categorías Pc, Pd, Ps, Pe, Pi, Pf y Po.
Sm	Símbolo, matemático
Sc	Símbolo, divisa
Sk	Símbolo, modificador
So	Símbolo, otro
S	Todos los símbolos. Esto incluye las categorías Sm, Sc, Sk y So.
Zs	Separador, espacio
Zl	Separador, línea
Zp	Separador, párrafo
Z	Todos los caracteres separadores. Esto incluye las categorías Zs, Zl y Zp.
Cc	Otro, control
Cf	Otro, formato
Cs	Otro, suplente
Co	Otro, uso privado
Cn	Otro, no asignado (ningún carácter tiene esta propiedad)
C	Todos los caracteres de control. Esto incluye las categorías Cc, Cf, Cs, Co y Cn.

Es posible obtener la categoría Unicode de cualquier carácter pasando dicho carácter al método **GetUnicodeCategory**.
Ejemplo:

```
Regex.Escape(caracter.ToString()),Char.GetUnicodeCategory(caracter));
```

Veamos a continuación la tabla correspondiente a los bloques con nombre:

Bloques con Nombre (Tabla de Microsoft)

.NET Framework proporciona los bloques con nombre que se muestran en la tabla siguiente. El conjunto de bloques con nombre compatibles está basado en Unicode 4.0 y Perl 5.6.

Intervalo de código	Nombre del bloque
0000 - 007F	**IsBasicLatin**

0080 - 00FF	**IsLatin-1Supplement**
0100 - 017F	**IsLatinExtended-A**
0180 - 024F	**IsLatinExtended-B**
0250 - 02AF	**IsIPAExtensions**
02B0 - 02FF	**IsSpacingModifierLetters**
0300 - 036F	**IsCombiningDiacriticalMarks**
0370 - 03FF	**IsGreek** **O bien** **IsGreekandCoptic**
0400 - 04FF	**IsCyrillic**
0500 - 052F	**IsCyrillicSupplement**
0530 - 058F	**IsArmenian**
0590 - 05FF	**IsHebrew**
0600 - 06FF	**IsArabic**
0700 - 074F	**IsSyriac**
0780 - 07BF	**IsThaana**
0900 - 097F	**IsDevanagari**
0980 - 09FF	**IsBengali**
0A00 - 0A7F	**IsGurmukhi**
0A80 - 0AFF	**IsGujarati**
0B00 - 0B7F	**IsOriya**
0B80 - 0BFF	**IsTamil**
0C00 - 0C7F	**IsTelugu**
0C80 - 0CFF	**IsKannada**
0D00 - 0D7F	**IsMalayalam**
0D80 - 0DFF	**IsSinhala**
0E00 - 0E7F	**IsThai**
0E80 - 0EFF	**IsLao**
0F00 - 0FFF	**IsTibetan**
1000 - 109F	**IsMyanmar**
10A0 - 10FF	**IsGeorgian**
1100 - 11FF	**IsHangulJamo**
1200 - 137F	**IsEthiopic**
13A0 - 13FF	**IsCherokee**
1400 - 167F	**IsUnifiedCanadianAboriginalSyllabics**
1680 - 169F	**IsOgham**
16A0 - 16FF	**IsRunic**
1700 - 171F	**IsTagalog**
1720 - 173F	**IsHanunoo**
1740 - 175F	**IsBuhid**
1760 - 177F	**IsTagbanwa**
1780 - 17FF	**IsKhmer**
1800 - 18AF	**IsMongolian**
1900 - 194F	**IsLimbu**
1950 - 197F	**IsTaiLe**
19E0 - 19FF	**IsKhmerSymbols**
1D00 - 1D7F	**IsPhoneticExtensions**
1E00 - 1EFF	**IsLatinExtendedAdditional**
1F00 - 1FFF	**IsGreekExtended**
2000 - 206F	**IsGeneralPunctuation**
2070 - 209F	**IsSuperscriptsandSubscripts**
20A0 - 20CF	**IsCurrencySymbols**
20D0 - 20FF	**IsCombiningDiacriticalMarksforSymbols** **O bien** **IsCombiningMarksforSymbols**
2100 - 214F	**IsLetterlikeSymbols**
2150 - 218F	**IsNumberForms**
2190 - 21FF	**IsArrows**
2200 - 22FF	**IsMathematicalOperators**
2300 - 23FF	**IsMiscellaneousTechnical**
2400 - 243F	**IsControlPictures**
2440 - 245F	**IsOpticalCharacterRecognition**

2460 - 24FF	**IsEnclosedAlphanumerics**
2500 - 257F	**IsBoxDrawing**
2580 - 259F	**IsBlockElements**
25A0 - 25FF	**IsGeometricShapes**
2600 - 26FF	**IsMiscellaneousSymbols**
2700 - 27BF	**IsDingbats**
27C0 - 27EF	**IsMiscellaneousMathematicalSymbols-A**
27F0 - 27FF	**IsSupplementalArrows-A**
2800 - 28FF	**IsBraillePatterns**
2900 - 297F	**IsSupplementalArrows-B**
2980 - 29FF	**IsMiscellaneousMathematicalSymbols-B**
2A00 - 2AFF	**IsSupplementalMathematicalOperators**
2B00 - 2BFF	**IsMiscellaneousSymbolsandArrows**
2E80 - 2EFF	**IsCJKRadicalsSupplement**
2F00 - 2FDF	**IsKangxiRadicals**
2FF0 - 2FFF	**IsIdeographicDescriptionCharacters**
3000 - 303F	**IsCJKSymbolsandPunctuation**
3040 - 309F	**IsHiragana**
30A0 - 30FF	**IsKatakana**
3100 - 312F	**IsBopomofo**
3130 - 318F	**IsHangulCompatibilityJamo**
3190 - 319F	**IsKanbun**
31A0 - 31BF	**IsBopomofoExtended**
31F0 - 31FF	**IsKatakanaPhoneticExtensions**
3200 - 32FF	**IsEnclosedCJKLettersandMonths**
3300 - 33FF	**IsCJKCompatibility**
3400 - 4DBF	**IsCJKUnifiedIdeographsExtensionA**
4DC0 - 4DFF	**IsYijingHexagramSymbols**
4E00 - 9FFF	**IsCJKUnifiedIdeographs**
A000 - A48F	**IsYiSyllables**
A490 - A4CF	**IsYiRadicals**
AC00 - D7AF	**IsHangulSyllables**
D800 - DB7F	**IsHighSurrogates**
DB80 - DBFF	**IsHighPrivateUseSurrogates**
DC00 - DFFF	**IsLowSurrogates**
E000 - F8FF	**IsPrivateUse o IsPrivateUseArea**
F900 - FAFF	**IsCJKCompatibilityIdeographs**
FB00 - FB4F	**IsAlphabeticPresentationForms**
FB50 - FDFF	**IsArabicPresentationForms-A**
FE00 - FE0F	**IsVariationSelectors**
FE20 - FE2F	**IsCombiningHalfMarks**
FE30 - FE4F	**IsCJKCompatibilityForms**
FE50 - FE6F	**IsSmallFormVariants**
FE70 - FEFF	**IsArabicPresentationForms-B**
FF00 - FFEF	**IsHalfwidthandFullwidthForms**
FFF0 - FFFF	**IsSpecials**

Después de haber visto las tablas anteriores se muestra un ejemplo de uso de \p.

El patrón:

\p{Sc}\d+\.\d+

Acepta las subcadenas resaltadas en amarillo:

23 $44.55 €34.68 429.22 £77.49

pues Sc representa los símbolos monetarios.

\P — Clase de Caracter de Negación de Unicode o de Bloque con Nombre

La expresión:

\P{name}

hace coincidir todos los caracteres que **no esten incluidos** en el bloque **name.**

Por lo anterior se infiere que esta expresión es la negación de **\p.**

Si se toma como ejemplo la expresión `\P{Sc}` estamos diciendo que serán aceptados todos los caracteres, **excepto los símbolos monetarios (**`$,€,£, etc.`**)**. Se debe observar que esta expresión viene a ser el equivalente al símbolo ^ dentro de un corchete.

4. Construcciones de Alternancia (Uso del OR lógico "|")

Las construcciones de alternancia crean un grupo de alternativas que provocaran o no una coincidencia dentro de la cadena en análisis.

Veremos a continuación los dos casos más usados:

- Caso 1

  Cuando el patrón es como sigue:

  es(te|ta|to)

  aquí se dice que que el prefijo de las subcadenas a aceptar debe ser "**es**", mientras que el sufijo debe ser "**te**", "**ta**" o "**to**".

  Y por tanto se aceptarían las subcadenas siguientes resaltadas en amarillo:

  `este 123 esta 456 esto`

- Caso 2

  Cuando el patrón es como sigue:

  (?(expresión_regular) sí | no)

donde sí es una expresión y no otra expresión regular. Esta contrucción de alternancia funciona de la manera siguiente basándonos en el ejemplo que sigue:

```
(?(\d)\d+|\x24%)
```

Si **expresión_regular** es de tipo **dígitos,** pues cada vez que entre un dígito se irá por la expresión **sí** (\d+), mientras que si no se cumple entonces la coincidencia se producirá según la expresión del **no** (**\x24%**) que indica que la subcadena tiene que ser un signo monetario ($) que es el que corresponde al ASCII 24 hexadecimal seguido de un símbolo de porciento.

Abajo aparecen resaltados en amarillo las subcadenas que dieron coincidencia dentro de la cadena en análisis:

123.44 123 abc $%

Se debe observar que la subcadena abc aunque no cumple la condición de estar compuesta por dígitos solamente pues no cumple el **no** ya que no está compuesta por $%.

Si modificamos el patrón anterior diciendo que en el **si** deben ir números decimales:

```
(?(\d)\d+\.\d+|\x24%)
```

entonces las coincidencias serían incluyendo el punto tal y como se muestra a continuación:

123.44123.44 abc $%

5. Los Corchetes "[]" más en detalle

Los corchetes dentro de las expresiones regulares tienen una gran importancia pues mediante estos se pueden realizar múltiples acciones que algorítmicamente pudieran representar mucho código si no se dispusiera de dichas expresiones.

Sirven para especificar por ejemplo a un conjunto de caracteres que pueden estar presentes en alguna posición específica de la cadena en análisis, también es posible especificar rangos bajo el uso del guión según vimos en ejemplos anteriores.

Dentro de los corchetes casi todos los metacaracteres pierden su significado especial y se toman como caracteres normales; solamente hay que anteponer la barra inversa a la propia barra inversa (se escribe "\\"), al corchete de cierre "]", al "^" si va al principio y al guión "-" si no va ni al principio ni al final. Por ejemplo, [\\|] acepta una barra inversa o una barra vertical "|". Tampoco se separan los caracteres con comas: [\\, |] aceptaría además la coma y el espacio. Dentro de los corchetes, el punto representa un carácter literal y no uno especial, por lo que no es necesario antecederlo con la barra inversa.

5.1 El signo +

El uso del signo + es de gran uso dentro de las expresiones regulares y significa "una o más repeticiones del elemento anterior". Para que se entienda bien se muestran a continuación algunos ejemplos de su uso:

1) Ejemplo:
Chequeo de los casos con punto al inicio sin signo:
Ejemplo: (.15, .972, etc.)
PATRON:
"^[.]\d+$"

2) Ejemplo:
Chequeo de los casos con punto al final sin signo:
Ejemplo: (15., 972., etc.)
"^\d+[.]$"

3) Ejemplo:
Chequeo de los casos con signo y formato {+,-}9(n).9(n):
Ejemplo: (+121.151, -45378.97277, etc.)
"^[+-]\d+\.?(\d+)?$"

4) Ejemplo:
Chequeo de los casos sin signo y formato 9(n).9(n)
y también de los enteros sin signo 9(n)
Ejemplo: 459.15, 262.9725, 673, etc.)
"^\d+\.?(\d+)?$"

6. Los paréntesis "()"

Los paréntesis son utilizados en las expresiones regulares en forma bastante parecida a los corchetes, aunque con las diferencias siguientes:

- Pueden ser usados para agrupar alternativas, por ejemplo:

La expresión "v1 (mayor que|menor que|igual a) v2" que se verá a continuación en una tabla evaluativa ilustra la forma de uso de los paréntesis:

Ejemplo1:

Importante:
Las subcadenas en amarillo dentro de la cadena en análisis significan que machean con la expresión regular, mientras que los caracteres no resaltados significan que no machean. En C# cuando sucedan casos similares al (4) y el (5) la cadena completa se da como ACEPTADA, sin embargo, cuando no exista ninguna subcadena que cumpla la condición, entonces el método IsMatch retorna false.

Cadenas Aceptadas	Cadenas No Aceptadas
1) v1 igual a v2	v1 igual a v3
2) v1 mayor que v2	v1 es mayor que v2

3) v1 menor que v2	v1 es menor que la variable v2
4) v1 igual a v2 ó v1 mayor que v2	v1 igual que v2 ó v1 mayor o igual que v2
5) v1 menor que v2 ó v1 mayor que v2	v1 desigual de v2

- En este otro ejemplo la expresión:

"^v1 (mayor que|menor que|igual a) v2$" restringue las posibilidades de la expresión anterior pues al insertar los caracteres **^ y $** de inicio y fin de cadena se está diciendo que los ejemplos (4) y (5) de la tabla anterior no serían aceptados.

- Los paréntesis también pueden ser utilizados con la finalidad de agrupar caracteres especiales dentro de los mismos pero sin que pierdan su significado, a diferencia de los corchetes. Ilustremos con ejemplos este concepto.

  Escojamos el siguiente patrón:

  (.A\d) lo cual indica que las cadenas aceptadas serán aquellas que contengan un carácter cualquiera (el punto sin barra inversa representa cualquier carácter), seguido de una letra "A" mayúscula (las expresiones regulares distinguen las mayúsculas de las minúsculas) y a continuación un dígito decimal. Si se quisiera un punto literal habría que escribir \. y para aceptar también la "a" minúscula, [Aa].

Ejemplo2:

Cadenas Aceptadas	Cadenas No Aceptadas
1) .A9	.a9 (la "a" minúscula no coincide con "A")
2) .A95 (contiene .A9)	.b9 (No tiene "A" como 2do caracter)
3) =A7 (el punto acepta cualquier carácter)	A7 (le falta el primer carácter)

Escojamos ahora el patrón anterior con corchetes: **[.A\d] (sin comas entre los caracteres: [.,A,\d] aceptaría también la coma)**

Este patrón, sin embargo, nos dice que machean todas las subcadenas que contengan un punto, una letra "A" mayúscula o un dígito decimal en cualquier orden en que aparezcan.

Ejemplo3:

** Importante:*
Se observará que los corchetes actúan en forma diferente a los paréntesis a pesar de que ambos son metacaracteres de agrupación pues en este ejemplo se nota que las cadenas no son obligatoriamente de 3 caracteres de longitud como en el ejemplo 2, sino que pueden ser cadenas de cualquier número de elementos siempre que los mismos contengan puntos, letras A mayúsculas y dígitos decimales del 0 al 9. Claro está, siempre que no utilicemos las cotas izquierda y derecha de la cadena de entrada mediante los simbolos ^ y $ al inicio y fin del patrón.

Cadenas Aceptadas	Porqué son Aceptadas
.A95	Contiene el punto, la letra "A" y dígitos
.a4444444444jklmxyz	Contiene el punto, la letra "A" y dígitos
7666skl.bbb	Contiene punto y dígitos
.A999xyza5uuuu6912883	Contiene el punto, la letra "A" y dígitos
333,,,,,.#####aAAAAaFaa	Contiene dígitos y la letra "A" y un punto

- Las agrupaciones entre paréntesis pueden representar en ocasiones una **"etiqueta"** o **"punto de referencia"** que son interpretados por el motor de búsqueda a fin de realizar acciones más avanzadas donde se conjugan con otros caracteres e*speciales como se verá más adelante.*

7. El caracter "."

El caracter "." representa cualquier caracter en su lugar excepto el **\n** y es usado cuando se desea que los caracteres que le siguen concuerden exactamente en el mismo orden de aparición dentro de la cadena en análisis, es decir que si escojemos como patrón (.er), este aceptaría cadenas como las siguientes:

- comer
- hacer
- tener
- arder
- Si tengo que comer y cocer voy a tener que partirme en dos

Se observará que las subcadenas en amarillo machean con el patrón, sin embargo, es conveniente aclarar que en lugar del punto tiene que haber un caracter, por tanto, la cadena **er** no machea con el patrón escogido.
Es conveniente que compare el resultado que se obtiene con el patrón (.er) y con el patrón [.er] y podrá ver las diferencias entre ambas opciones, se lo dejamos a usted.

Si se quisiera aceptar también subcadenas **er** el patrón podría ser el siguiente:

```
(.er|er)      <== Usando OR lógico
```

y entonces se aceptarían cadenas como la siguiente:

ercomeraaatemerbbbsostener

8. El signo de interrogación "?"

El signo de interrogación en las expresiones regulares puede tener varios usos:

1. Puede expresar opcionalidad en la aparición de un caracter o de una subcadena dentro de la cadena en análisis, por ejemplo, si escojemos el patrón siguiente:

   (cen?tro)

   esto significaría que el caracter que precede al símbolo ? sería opcional dentro de la cadena y por ende serían aceptadas las subcadenas **cetro** o **centro**.

   * ***Es posible anidar los paréntesis cuando se desea chequear más de un caracter antes del caracter ?, en el ejemplo que sigue son aceptadas las cadenas que tengan "nxyz" o nada entre ce y tro.***

   ```
   Patrón ==>              (ce(nxyz)?tro)
   Cadena en análisis ==>  cenxyztroaaaaacetrobbbbb
   ```

2. Es posible también, basándonos en el ejemplo anterior, buscar subcadenas de más de un caracter, por ejemplo:

   Sept(iembre|ember)?

   esto significaría que las subcadenas **iembre** o **ember** que estan separadas por el OR lógico ("|") y que aparecen dentro de los parántesis que preceden al signo ?, serían opcionales dentro de la cadena.

   Por tal razón las subcadenas **Sept**, **Septiembre** o también **September** serían aceptadas por el patrón. Claro está que no serían aceptadas subcadenas como **Noviembre**, o **December.**

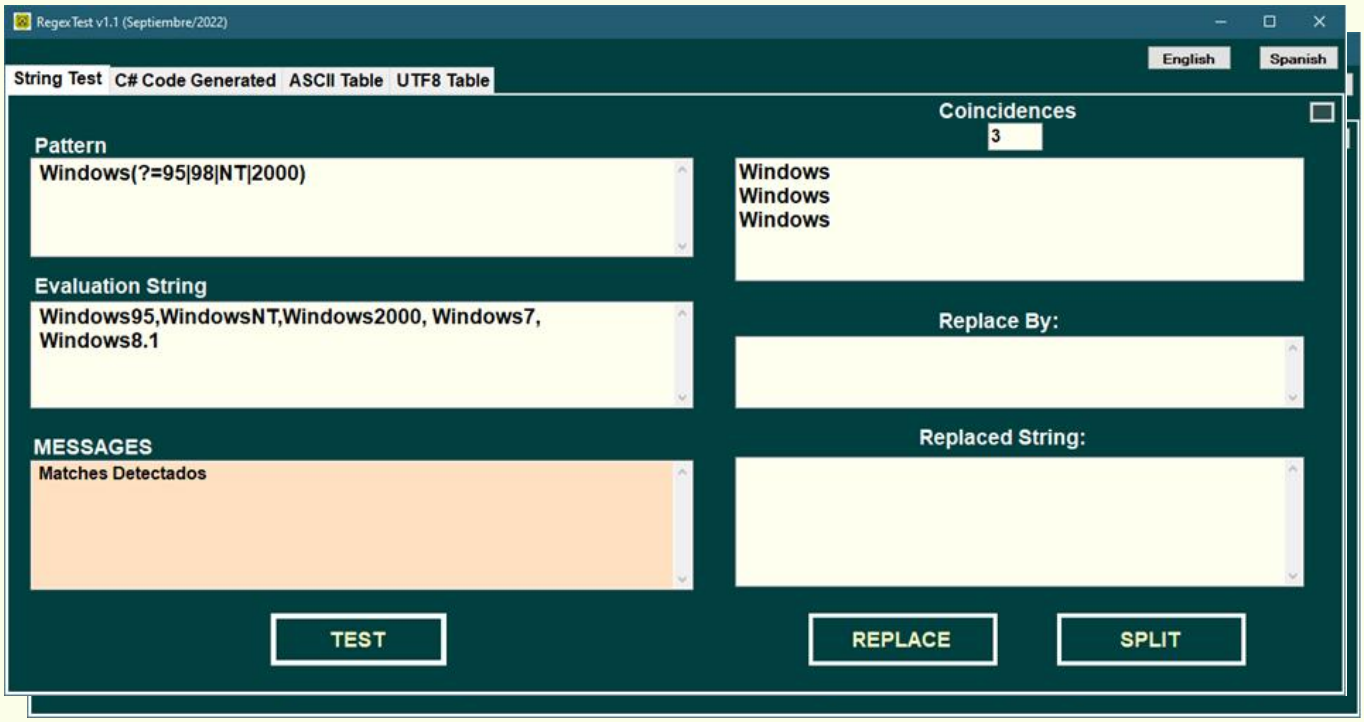
9. El formato (?:patron)

Este formato agrupa una parte del patrón sin guardarla como grupo capturado (por eso se le llama grupo sin captura) y se utiliza en muchas ocasiones vinculado al "|" (OR).

Si se escoje el siguiente patrón: **industr(?:y|ies),** esta sería una forma simplificada respecto a este otro patrón equivalente: **industry|industries.**

Si procesamos este patrón bajo nuestro utilitario RegexTest nos daría el siguiente resultado:

10. El formato (?=patron)



Este formato se conoce como búsqueda anticipada: comprueba que a continuación de la coincidencia venga lo indicado entre los paréntesis, pero sin incluirlo en la coincidencia. Se utiliza en muchas ocasiones vinculado al "|" (OR).

Por ejemplo:
Patrón ==> Windows(?=95|98|NT|2000)
Hace que la siguiente cadena tenga tres coincidencias (en amarillo):
Windows95,WindowsNT,Windows2000, Windows7, Windows8.1

*** Se observará que Windows7, y Windows8.1 no tienen coincidencias.
En RegexTest aparecería así:

11. El formato (?!patron)

Es la negación del anterior, por tanto, si se escribiera:
Windows(?!95|98|NT|2000)
Hace que la siguiente cadena tenga dos coincidencias, que son exactamente las que no fueron aceptadas en el ejemplo del epígrafe anterior:
Windows95,WindowsNT,Windows2000, Windows7, Windows8.1
Ejemplo:

12. Los Cuantificadores

Aunque ya se han visto anteriormente podemos resumir que a los simbolos:

?, ***, **+**, **{n,m}** o **{n,}**

se les conoce como cuantificadores en las expresiones regulares.

A continuación, se hace un resumen ilustrativo de diferentes casos a forma de ejemplo y vistos más en detalle:

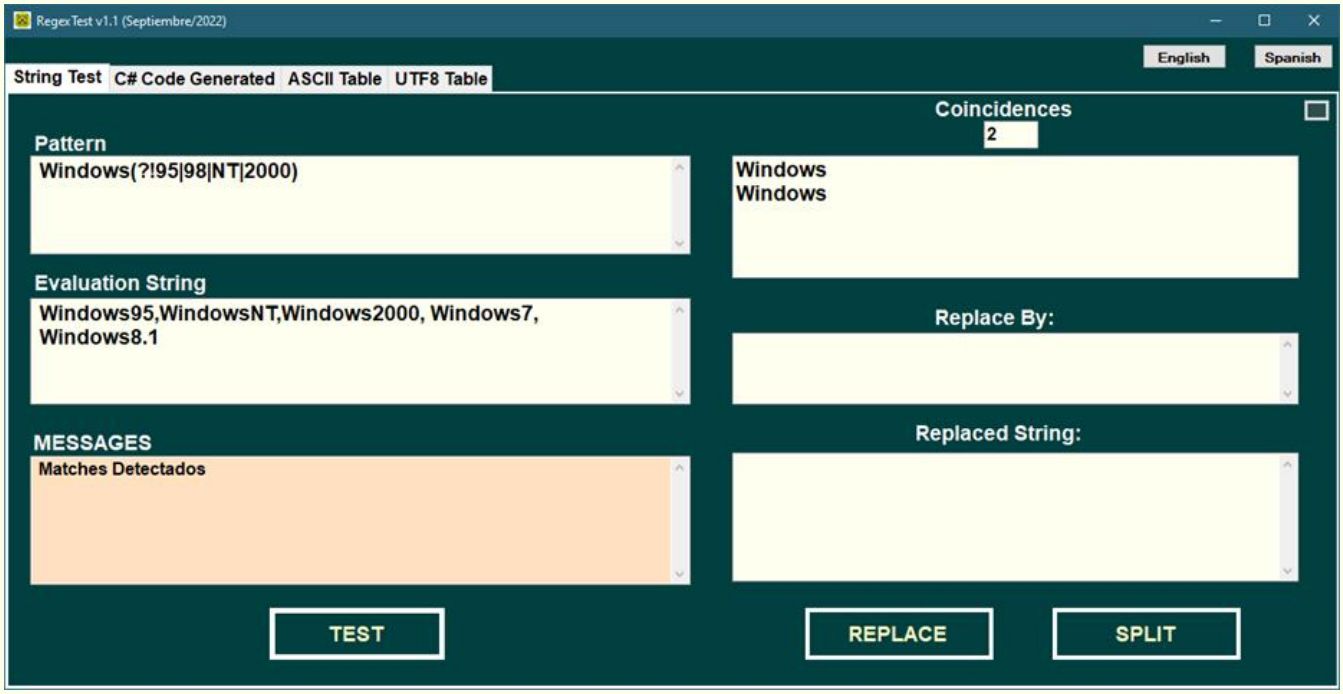
Cuantificadores expansivos y no expansivos

* Según Microsoft:

Los cuantificadores especifican cuántas instancias de un carácter, un grupo o una clase de caracteres debe haber en la entrada para que se encuentre una coincidencia. En la tabla siguiente se enumeran los cuantificadores admitidos por .NET Framework.

Las cantidades *n* y *m* son constantes de tipo entero. Normalmente, **los cuantificadores son expansivos** y hacen que el motor de expresiones regulares busque coincidencias con tantas apariciones de modelos concretos como sea posible.

Al anexar el caracter **?** a un cuantificador lo hace **no expansivo** y hace que el motor de expresiones regulares coincida con el mínimo de apariciones posible.



Cuantificador expansivo	Cuantificador no expansivo	Description
*	*?	Busca cero o más coincidencias.
+	+?	Busca una o varias coincidencias.
?	??	Busca cero o una coincidencia.
{n}	{n}?	Busca exactamente n coincidencias.
{n,}	{n,}?	Busca al menos n coincidencias.
{n,m}	{n,m}?	Busca de n a m coincidencias.

Algunos ejemplos del uso de cuantificadores (Microsoft)

Cuantificador	Descripción	Modelo	Coincidencias
*	Indica que puede haber 0 o más apariciones de dígitos antes del punto decimal y un solo dígito a la derecha del punto.	"\d*\.\d"	".5" acepta a ".5" "59.66" acepta a "59.6" "473.833.4" acepta a "473.8" y a "33.4" (2 coincidencias)
+	Indica que debe haber 1 o más apariciones de dígitos antes del punto decimal y un solo dígito a la derecha del punto. No se aceptan 0 apariciones como en el caso anterior (*)	"\d+\.\d"	".5" no se acepta "1.5" si se acepta
{n}	Coincide con el elemento anterior exactamente n veces.	",\d{3}"	",043,958,317" da 3 coincidencias
{n,}	Coincide con el patrón anterior(\d) al menos n veces.	"\d{2,}"	"166", "29", "1930"
{n,m}	Coincide con el patrón anterior (\d) al menos n veces, pero no más de m veces.	"\d{3,5}"	"166", "17668","19302" en "193024"
*?	Coincide con el patrón anterior (\d) cero o más veces, pero el menor número de veces que sea posible.	"\d*?\.\d"	".0", "19.9", "219.9"
+?	Coincide con el patrón anterior (be) una o más	"be+?"	"be" en "bebé"

	veces, pero el menor número de veces que sea posible.		
{n}?	Coincide con el patrón anterior (\d) exactamente n veces.	",\d{3}?"	",434" en ",434" ",434" en ",43488"
{n,}?	Coincide con el patrón anterior (\d) al menos n veces, pero el menor número de veces posible.	"\d{2,}?"	"166" con 1 coincidencia "1930" con dos coincidencias
{n,m}?	Coincide con el patrón anterior (\d) entre n y m veces, pero el menor número de veces posible.	"\d{2,5}?"	"19","30", y "24" en "193024" (3 coincidencias)

Patrón	Datos permitidos
a*x	x, ax, aax, aaax...
a?x	ax, x
a+x	ax, aax, aaax...
(a|b)+x	ax, bx, aax, abx, bax, bbx, aaax, aabx, abax, abbx, baax, babx, bbax, bbbx, aaaax...

13. Programas Utilitarios de Expresiones Regulares

El utilitartio **RegexTest** ha sido programado integramente en lenguaje C# como una aplicación de tipo **Windows Forms** y por tal razón todos los ejemplos dados en el presente documento han sido probados bajo este utilitario con la finalidad de evitar incompatibilidades que pueden existir respecto a otras plataformas, si el lector desea probar expresiones regulares en otras plataformas le sugerimos que pruebe el **RegexBuddy** que con respecto a otros productos similares se ve bastante robusto y aunque es de tipo **shareware** no se oferta a precio muy elevado.

Aunque anteriormente hemos mostrado los resultados haciendo referencia al screen de **RegexTest**, se mostrará a continuación sus características fundamentales.

La pantalla principal está concebida bajo un formato de tipo multitabs donde en el primer TAB se dan todos los parámetros que forman parte de la validación que se desea realizar (patrón o pattern, cadena a analizar (string), el número de coincidencias detectadas (matches), por quien se desea reemplazar a la cadena de entrada, la cadena reemplazada en los casos de reemplazamiento, y la división de una cadena en subcadenas o SPLIT).

En este TAB se tienen tres botones:

- **Test de Cadena**
- **Reemplazar**
- **Split**

Después de realizado el test de cadena se puede proceder a realizar reemplazamientos o división de cadenas en subcadenas específicas (split)

14. El método Replace de la clase **Regex**.

Hasta el momento hemos visto diferentes casos para encontrar subcadenas dentro de una cadena dada que sean aceptadas por un patrón determinado (método IsMatch de Regex), sin embargo, es posible también reemplazar las subcadenas coincidentes por una cadena de caracteres dada, y esto se realiza precisamente mediante el método **Replace** de la clase Regex que se explicará a continuación.

Método Replace

Este método posee varias sobrecargas que pueden ser vistas en el Help del Visual Studio, pero aquí se muestran dos de uso frecuente:

```
1)Regex.Replace(string Cadena, string Patron, string ReemplazarPor);
2)OBJ.Replace(string Cadena, string ReemplazarPor);
```

En el primer caso no es necesario instanciar ningun objeto pues se está usando el método **Replace** asociado a la clase `Regex`.

En el segundo caso se asume que previamente se ha instanciado al objeto OBJ con new:

```
Regex OBJ = new Regex(@"patrón de validación");
```

Ambas retornan un string como resultado de reemplazar las subcadenas que han hecho **Match** con el patrón dado por el string de la variable **ReemplazarPor**.

Por ejemplo, si suponemos que nuestro patrón es:

```
[\x61-\x7a]{3}?
```

el cual acepta todas las subcadenas de tres elementos que contengan letras minúsculas de la "a" hasta la "z", y la cadena que damos es:

```
aaa111bbb222ccc333ddd
```

donde las subcadenas que responden son las letras resaltadas en amarillo, entonces si ponemos:

```
string R = Regex.Replace("aaa111bbb222ccc333ddd",
                         "[\x61-\x7a]{3}?",
                         "ZZZ");
```

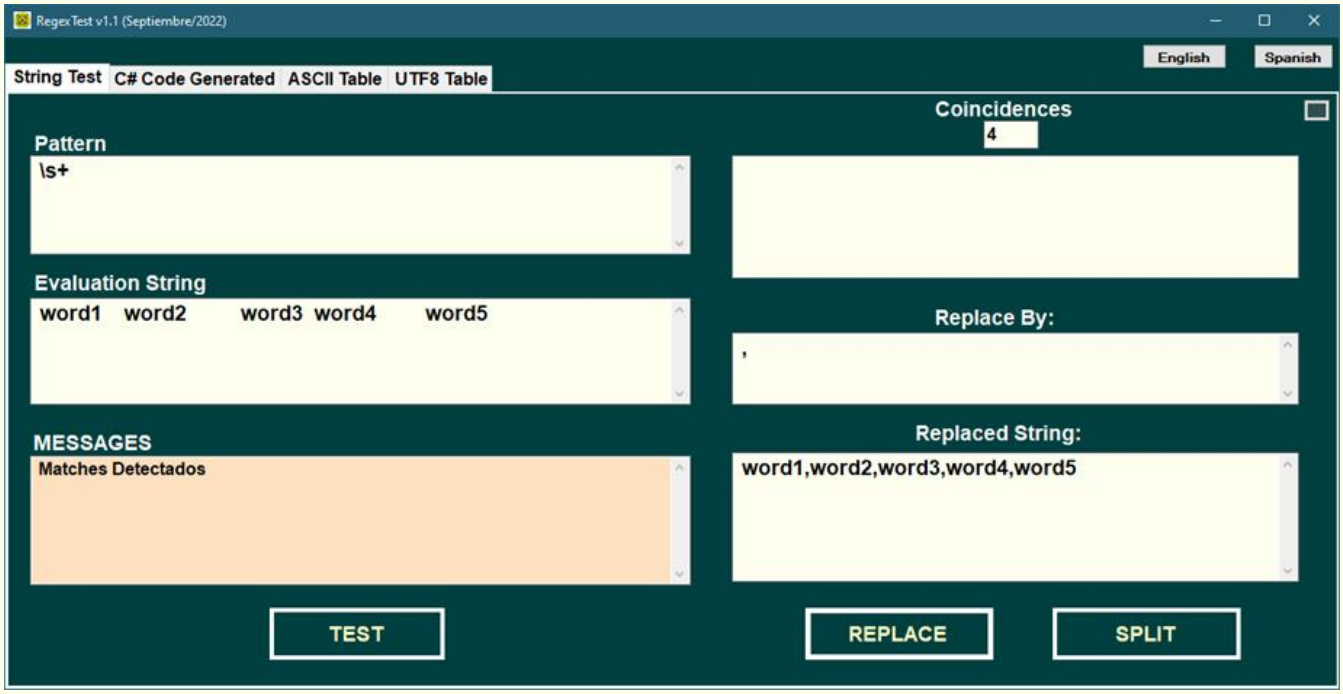
En **R** quedaría lo siguiente:

R = "ZZZ111ZZZ222ZZZ333ZZZ"

Veamos a continuación un ejemplo con un caso nuevo de patrón que aparece frecuentemente y es la detección de espacios lo cual se realiza mediante la secuencia de escape \s.

En el ejemplo que sigue se desean eliminar todos los espacios sobrantes que separan a word1, word2, etc.

Lo primero que debemos hacer es definir nuestro patrón de la forma siguiente:
\s+ lo cual quiere decir que se detectarán uno o varios espacios donde se encuentren (recuerde al argumento **+** que significa una o varias veces a diferencia del ***** que significa cero o varias veces).



Después le damos al **RegexTest** la cadena a evaluar que contiene múltiples espacios y tocamos inmediatamente después al botón **Test de Cadena** dando como resultado 4 coincidencias o matches que no aparecen reflejados bajo **Coincidencias** por ser espacios.

Para realizar un reemplazamiento, después de haber realizado el test se define cual va a ser el caracter, o la cadena que reemplazará a las coincidencias detectadas, que en nuestro caso es la coma, y al dar click en el botón de **Reemplazar** quedará el resultado en el box **Cadena Reemplazada.**

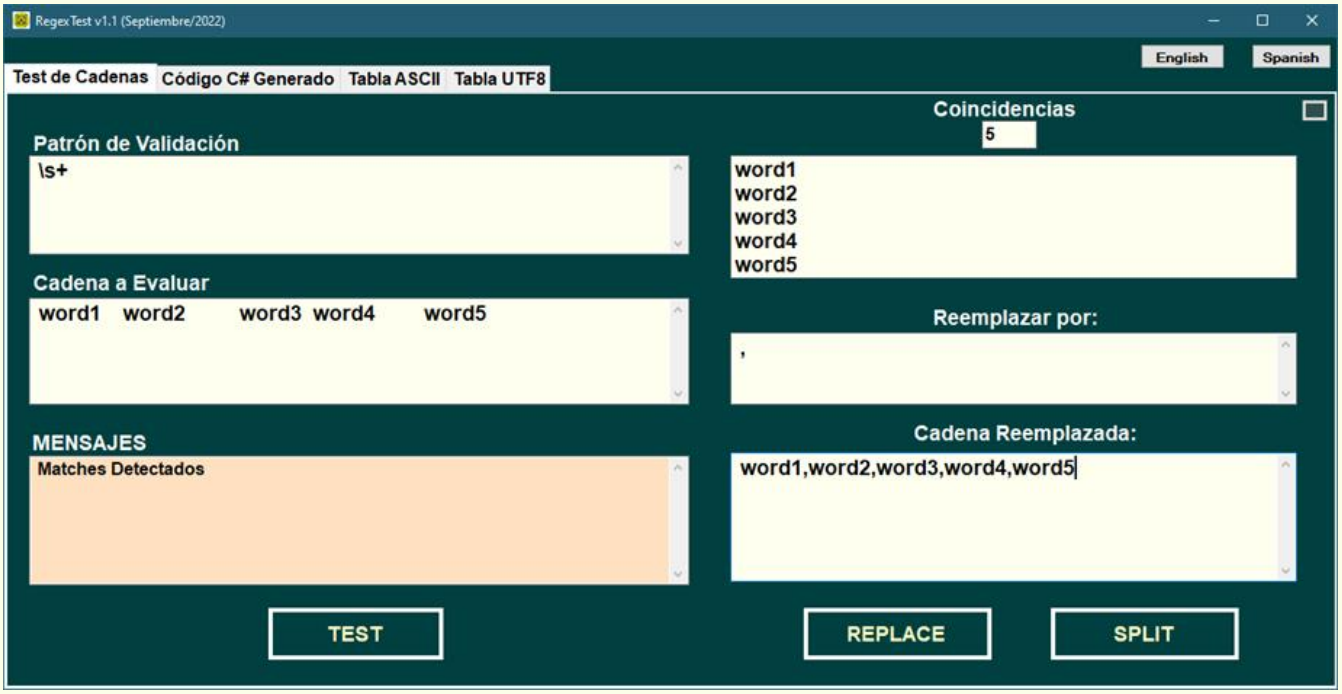
Si después de haber realizado todas las acciones anteriores queremos ver el código C# correspondiente a las mismas, pues esto es perfectamente posible mediante el utilitario **RegexTest** y además muy sencillo, pues basta con tocar el segundo TAB denominado **Código C# Generado** y le aparecerá en un box editable todo el código generado que corresponde a estas acciones como sigue:

```
   //=========================================================================
   //                         Código de IsMatch()
   //=========================================================================
       if  (Regex.IsMatch(@"word1      word2  word3 word4             word5",@"\s+"))
           {
           // Poner aquí código de si SE CUMPLE el match
           }
       else
           {
           // Poner aquí código de si NO SE CUMPLE el match
           }
   // A continuación se obtiene en la variable CoincidenciasN la cantidad
   // de Matches qie se obtuvieron
       int Coincidencias1 = Regex.Matches(@"word1      word2  word3 word4
 word5",@"\s+").Count;
   //=========================================================================
   //                         Código de Replace()
   //=========================================================================
   string CadenaReemplazada1 = Regex.Replace(@"word1      word2  word3 word4
 word5",@"\s+",@",");
   //=========================================================================
```

Como se observa, al programador le queda en la mano en código C# correctamente generado y puede copiarlo y pegarlo en su programa haciéndole las adecuaciones que desee con mucha facilidad pero sin el temor de que pueda presentar errores de sintáxis en su programa.

Ahora bien, si en vez de reemplazar se hubiese deseado hacer una separación de todas las palabras del ejemplo (word1, word2, etc.), cuyo separador es en nuestro caso los espacios originales que se cambiaron por comas, pues bastaría con tocar el botón **Split** generándose el código siguiente:

```
  //=========================================================================
  //                         Código del Split()
  //=========================================================================
  // A continuación se obtienen las subcadenas que se encuentran entre las
  // subcadenas que coincidieron en el MATCH asumiendo que estas son los
  // delimitadores dentro de la cadena en análisis.
  //=========================================================================
      string[] CadenaSplit1 = Regex.Split(@"word1      word2  word3 word4
word5",@"\s+");
      for (int IDX = 0; IDX < CadenaSplit1.Length; IDX++)
           {
           listBox1.Items.Add(CadenaSplit1[IDX]);
           }
  //=========================================================================
```



La pantalla principal de **RegexTest** se mostraría de la forma siguiente:
Después de haber visto el proceso de los casos anteriores debemos hacer énfasis en aclarar la gran ventaja del utilitario RegexTest y es que en la medida en que se fueron ejecutando dichos casos anteriores pues en el TAB denominado: **Código C# Generado**
Aparece el código de programa correspondiente a cada caso, el cual se va generando dinámicamente en la medida en que damos **click** en **TEST, REPLACE o SPLIT** tal y como se muestra a continuación:

```
//======================================================================
//                        USING's for Regex
//                  < Generated Code By RegexTest >
//======================================================================
// The code generated by RegexTest shown below has as objetive to provide
// you with a template that can be easily modified by you and be inserted
// in any region of your program.
// This template assumes the existence of two textBox named:
// - CADENA.Text
// - PATRON.Text
// It also assumes the existence of a listBox named:
// - <listBox1>
// For this reason, if you have the string to parse in a variable of type
// string, like the pattern, it will suffice to do a simple search with
// replacement so that the implicit names of the textBox above are replaced
// by the names of the variables you want
//======================================================================
    using System.Text.RegularExpressions;
//======================================================================
```

```
  //=========================================================================
  //                          Código de IsMatch()
  //=========================================================================
      if  (Regex.IsMatch(@"industria industrias industry industries",@"industr(?:y|ies)"))
          {
          // Poner aquí código de si SE CUMPLE el match
          }
      else
          {
          // Poner aquí código de si NO SE CUMPLE el match
          }
  // A continuación se obtiene en la variable CoincidenciasN los Matches Obtenidos
      int Coincidencias1 = Regex.Matches(@"industria industrias industry
industries",@"industr(?:y|ies)").Count;
  //=========================================================================
  //                          Código de IsMatch()
  //=========================================================================
      if  (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)"))
          {
          // Poner aquí código de si SE CUMPLE el match
          }
      else
          {
          // Poner aquí código de si NO SE CUMPLE el match
          }
  // A continuación se obtiene en la variable CoincidenciasN los Matches Obtenidos
      int Coincidencias2 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)").Count;
  //=========================================================================
  //                          Código de IsMatch()
  //=========================================================================
      if  (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)"))
          {
          // Poner aquí código de si SE CUMPLE el match
          }
      else
          {
          // Poner aquí código de si NO SE CUMPLE el match
          }
  // A continuación se obtiene en la variable CoincidenciasN los Matches Obtenidos
      int Coincidencias3 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)").Count;
  //=========================================================================
  //                          Código de IsMatch()
  //=========================================================================
      if  (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?!95|98|NT|2000)"))
          {
          // Poner aquí código de si SE CUMPLE el match
          }
      else
          {
          // Poner aquí código de si NO SE CUMPLE el match
          }
  // A continuación se obtiene en la variable CoincidenciasN los Matches Obtenidos
      int Coincidencias4 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?!95|98|NT|2000)").Count;
```

```
  //=========================================================================
  //                          Código de IsMatch()
  //=========================================================================
      if  (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
           Windows8.1",@"Windows(?!95|98|NT|2000)"))
          {
          // Poner aquí código de si SE CUMPLE el match
          }
      else
          {
          // Poner aquí código de si NO SE CUMPLE el match
          }
  // A continuación se obtiene en la variable CoincidenciasN los Matches Obtenidos
      int Coincidencias5 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
                Windows8.1",@"Windows(?!95|98|NT|2000)").Count;
  //=========================================================================
  //                          Código de IsMatch()
  //=========================================================================
      if  (Regex.IsMatch(@"word1 word2 word3 word4 word5",@"\s+"))
          {
          // Poner aquí código de si SE CUMPLE el match
          }
      else
          {
          // Poner aquí código de si NO SE CUMPLE el match
          }
  // A continuación se obtiene en la variable CoincidenciasN los Matches Obtenidos
      int Coincidencias6 = Regex.Matches(@"word1 word2 word3 word4 word5",@"\s+").Count;
  //=========================================================================
  //                          Código de Replace()
  //=========================================================================
      string CadenaReemplazada1 = Regex.Replace(@"word1 word2 word3 word4 word5",@"\s+",@",");
  //=========================================================================
  //=========================================================================
  //                          Código del Split()
  //=========================================================================
  // El método Split() extrae todas las subcadenas no contenidas en las subcadenas
  // de coincidencias
  //=========================================================================
      string[] CadenaSplit1 = Regex.Split(@"word1    word2          word3  word4
word5",@"\s+");
      for (int IDX = 0; IDX < CadenaSplit1.Length; IDX++)
           {
           listBox1.Items.Add(CadenaSplit1[IDX]);
           }
  //=========================================================================
```

La ventaja de disponer de este utilitario es que elimina la posibilidad de que el programador cometa errores de sintáxis en su programa, al tener la posibilidad de insertar el código generado en los lugares que desee.

15. Exclusión de Grupos de Caracteres de una Clase de Caracteres

Estas expresiones se utilizan para quitar o excluir un conjunto de caracteres de una clase de caracteres, el formato que se utiliza es el siguiente:

[grupo_base-[grupo_excluido]]

donde son obligatorios los corchetes y el guión son obligatorios en la expresión.

Ejemplo:

[0-9-[01]]

Este patrón aceptaria las siguientes subcadenas, pues **no se acepta ni el cero ni el uno**:

1234 000 544 666 999 111

16. Ejercicios a Resolver

A continuación, aparecen una serie de ejercicios prácticos que le ayudarán en la asimilación de las expresiones regulares en C#.

Es aconsejable que utilice el RegexTest para validar resultados.

16.1 - Ejercicio1

Crear un patrón de expresión regular que acepte Nombres y Apellidos en español. (Pueden contener vocales acentuadas y la ñ. La primera letra debe estar en mayúscula)

16.2 - Ejercicio2

Crear un patrón de expresión regular que acepte números decimales de a lo más cinco dígitos enteros y dos decimales, sin signo.

16.3 - Ejercicio3

Crear un patrón de expresión regular que acepte números decimales de a lo más cinco dígitos enteros y dos decimales obligatorios, con signo eventualmente.

16.4 - Ejercicio4

Crear un patrón de expresión regular que acepte números telefónicos con el siguiente formato:

ddd-ddd-dddd

16.5 - Ejercicio5

Crear un patrón de expresión regular que acepte solamente líneas en blanco.

16.6 - Ejercicio6

Crear un patrón de expresión regular que acepte fechas con el siguiente formato:

dd/mm/aaaa

Se asume que los días deben estar en el intervalo del 1 al 31, los meses del 1 al 12 y el año debe estar entre 1900 y 2099. (No analizar si el año es bisiesto ni tampoco los meses de 30 días pues esto normalmente requiere de un pequeño algoritmo)

16.7 - Ejercicio7

Crear un patrón de expresión regular que acepte palabras que representen verbos en lengua española (terminación **ar, er o ir).**

16.8 - Ejercicio8

Crear un patrón de expresión regular que acepte un número de identificación de un centro de trabajo cuya ley de formación es la siguiente:

- Los dos primeros caracteres deben ser letras minúsculas desde la "a" hasta la "z".
- A continuación le sigue un guión.
- Finalmente deben haber 5 dígitos decimales que deben estar en el intervalo del 4 al 8.

Por ejemplo: "ab-47856"

16.9 - Ejercicio9

Crear un patrón de expresión regular que acepte cadenas que contengan tríos de letras minúsculas.

Es recomendable que se consulte al Help de Visual Studio haciendo búsqueda de "Expresiones Regulares" y encontrará múltiples ejemplos y conceptos que le ayudarán a profundizar en este tema.

Resumen Final

Lo visto hasta el momento ha tenido como objetivo principal el dar una visión elemental de las **expresiones regulares** y ha sido orientado en su totalidad a los programadores no familiarizados con el tema.

Aunque el presente documento ha sido previsto solamente para programadores del lenguaje C# de Microsoft, puede ser útil también para cualquier programador de otros lenguajes de .NET, como es, por ejemplo, el caso de Visual Basic, C++, etc, e incluso, de otras plataformas y lenguajes. Esto se debe a que las ideas básicas de las expresiones regulares son comunes a muchos lenguajes de programación, editores de texto, etc., aunque cada uno tiene sus pequeñas diferencias.

Si el lector desea continuar profundizando en temas avanzados de las expresiones regulares, es conveniente que estudie el Help de Microsoft para Visual Studio con el objetivo de asimilar los diferentes conceptos que aquí se enseñan.

Esperamos que este documento introductorio le haya sido útil, buen provecho!.

El Autor